

Deep Dive — Reliability, Observability, Security, Deployment, Cost, Capstone

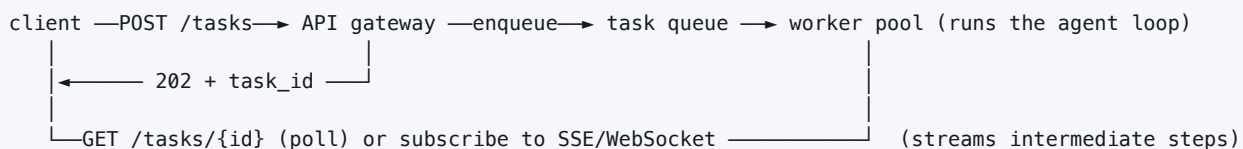
Part 10 — AI Architecture · 10-ai-architecture/notes/02-production-patterns.md

Everything in notes/01-reference-architecture.md [10-ai-architecture/notes/01-reference-architecture.md] draws the boxes. This note is about what makes the boxes survive contact with real traffic, real attackers, and a real finance review.

1. Scalability and reliability patterns

1.1 Async/queue-based processing for long-running agent tasks

A chat turn fits inside an HTTP request/response. A multi-step agent task (research a topic, run a 20-tool-call workflow, wait on a human approval) does not — it can run for minutes, and holding an HTTP connection open for minutes is how you learn about your load balancer's idle-timeout the hard way.



Submit the task, return a task ID immediately, and let the client poll or subscribe for progress. The queue (SQS, Redis Streams, a Postgres-backed job table — pick based on throughput and durability needs, not fashion) is also your natural point for **backpressure**: if workers can't keep up, the queue grows instead of the system falling over, and you get a real metric (queue depth) to alert on instead of discovering the problem via latency complaints.

1.2 Idempotency for retries

Every retry — client retry on timeout, queue redelivery, orchestration-layer retry after a transient provider failure — replays a request. If that request has a side effect (charge a card, send an email, create a ticket), a retry without an idempotency key duplicates the side effect.

Pattern: every request that can have a side effect carries a client-generated idempotency key ((thread_id, step_id, request_id) is a good composite). The orchestration layer checks a durable store for that key before executing; if seen, it returns the memoized result instead of re-running the action.

This is not hypothetical for agent systems — it is the default failure mode. LangGraph's resume semantics re-run an interrupted node *from the top*, not from where it paused, which means a node that calls `send_email()` before waiting on a human approval will send the email again on every resume. See the re-execution trap [10-ai-architecture/16-ai-tech-stack/tracks/langgraph/notes/02-production-craft.md]

for the exact mechanism and the fix (side effects in their own memoized node/task). The architectural rule that generalizes beyond LangGraph: **design every node/step so that running it twice is boring.**

1.3 Circuit breakers and graceful degradation

Covered mechanically in notes/01-reference-architecture.md §3 [10-ai-architecture/notes/01-reference-architecture.md] and implemented in code/model_gateway_simulator.py [10-ai-architecture/code/model_gateway_simulator.py]. The architectural point: a circuit breaker is worthless unless something sensible happens when it's open. Define the degradation path *before* you need it:

Failure	Degraded behavior (not a hard failure)
Primary model provider down	Failover to secondary provider (possibly a smaller/cheaper model — worse answers beat no answers)
Vector DB unreachable	Answer from the model's parametric knowledge with a visible "retrieval unavailable" flag, instead of a 500
Reranker service down	Skip reranking, serve raw retrieval order — quality dips, availability doesn't
Budget circuit breaker tripped	Route to the cheapest tier only, or serve a cached/canned response, instead of stopping the product
Guardrail/moderation service down	Fail closed, not open — this is the one row where "degrade" means "refuse," see §3

Graceful degradation is a business decision disguised as an engineering pattern — decide per failure mode whether "worse" or "unavailable" is the safer default, and write it down.

1.4 Multi-region / multi-provider redundancy

Redundancy has two independent axes and teams often build only one: - **Multi-region**: protects against a cloud region outage; requires your data layer (vector DB, cache, conversation store) to be replicated or regionally independent, not just your compute. - **Multi-provider**: protects against a single model vendor's outage or a rate-limit wall during a traffic spike; this is what the model gateway's failover (§ above, and Module 10's gateway code) exists for.

You need multi-provider before you need multi-region for almost every product — a provider outage is far more common and far more within your control to mitigate than a full cloud region loss.

1.5 Streaming architecture (SSE / WebSocket)

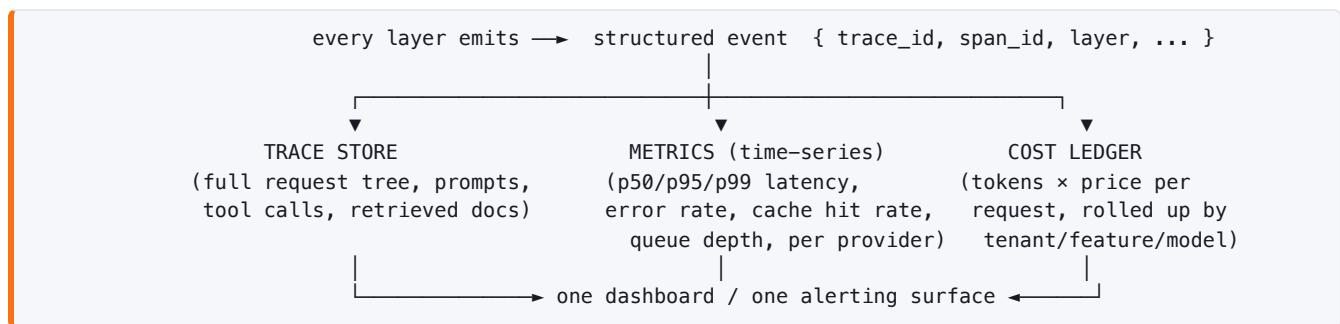
Users perceive latency as "time to first token," not "time to complete response" — stream. Two options:

	Server-Sent Events (SSE)	WebSocket
Direction	server → client only	bidirectional
Fits	token streaming, progress updates	token streaming and mid-generation client interrupts, multi-turn tool-approval UIs
Infra	plain HTTP, works through most proxies/CDNs unmodified	needs a persistent connection, stickier load balancing
Reconnect	automatic in-browser (EventSource), with Last-Event-ID resume	you build resume logic yourself

Opinion: default to SSE. Reach for WebSocket only when the client genuinely needs to send events mid-stream (barge-in, live tool-approval), because the operational simplicity of SSE is worth a lot. Either way, the streaming connection terminates at the **API gateway/BFF**, not at the orchestration layer directly — the gateway is what lets you swap the orchestration implementation without a client protocol change.

2. Observability architecture — one concern, not four bolted-on tools

Tracing, logging, metrics, and cost dashboards are usually built by four different people at four different times, in response to four different incidents. That is the wrong order. They are one architecture concern — "can I answer 'what happened, to whom, at what cost, and why' for any request after the fact" — and should be designed together, before launch.



2.1 What to instrument BEFORE launch, not after the first incident

- **Every model call:** prompt hash (not raw prompt, for cost — but see the security note below), model + version, input/output token counts, latency, cost, cache hit/miss, which layer initiated it.
- **Every tool/retrieval call:** which tool, arguments (redacted if sensitive), latency, success/failure, and — for RAG — which documents were retrieved and their relevance scores.
- **Every request end-to-end:** a single `trace_id` propagated through gateway → orchestration → model → tool layers, so one incident produces one queryable tree instead of four teams grepping four logs.
- **A cost ledger keyed by tenant/feature**, updated synchronously enough to drive a hard budget breaker (\$5) — a cost dashboard that lags by a day cannot prevent the bill it reports on.

- **Eval hooks:** log enough of the input/output to replay a request into your eval suite (Module 15 [10-ai-architecture/15-ai-evals]) later. The traces you capture *are* your future regression test corpus.

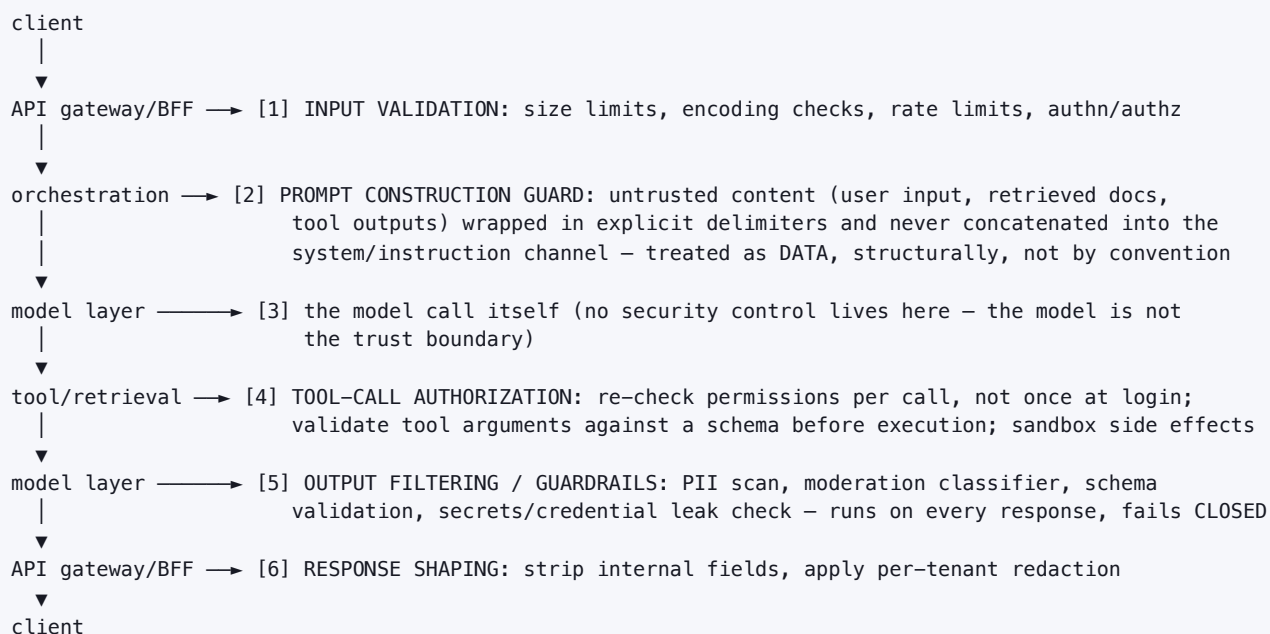
The reason this must exist before launch: the first production incident is exactly when you have zero historical data to diagnose it with, unless you already logged the thing that broke. Retrofitting observability after an incident means you've paid for the incident and still can't fully explain it.

2.2 Where this lives, and the link

Full trace-collection mechanics, span naming, and the LLM-specific tracing patterns (this is a large enough topic to deserve its own module) live in Module 16 / langsmith [10-ai-architecture/16-ai-tech-stack/tracks/langsmith/notes/01-core-concepts.md]. The architectural point here is placement: observability is not a box in the layer diagram, it is instrumentation *inside every box*, wired to one collection point, from the first commit.

3. Security architecture — where each control lives in the request path

Security controls that exist only as "a paragraph in the system prompt" are not controls — they are requests the model can ignore or be talked out of. This module places each control on the actual request path; the mechanisms (prompt injection techniques, jailbreak taxonomies, red-teaming methodology) are Module 13 [10-ai-architecture/13-ai-security]'s job, not this one's.



Five things worth saying explicitly:

1. **Untrusted content is anything the model didn't generate and you don't control** — user input, a retrieved document, a tool's return value, another agent's message in a multi-agent system. All of it gets

delimited and treated as data at the orchestration layer (control 2), because that is the only layer that knows what's untrusted before it's assembled into a prompt.

2. **Guardrails are not one box** — input-side (block obviously malicious input before it costs a model call) and output-side (control 5) are different controls with different jobs; input validation is cheap and coarse, output filtering is your last line and must be conservative.
3. **Fail closed on the guardrail path.** If the moderation/output-filter service is unreachable, that is the one place in the whole architecture where "degrade gracefully" is wrong — refuse the response instead of serving unfiltered content, unlike every other graceful-degradation row in §1.3.
4. **Re-authorize at the tool layer, not just at login.** A long-running agent session can outlive a permission change; checking authz only at the gateway means a revoked permission doesn't take effect until the next login.
5. **The model itself is never the trust boundary.** Every defense above is designed assuming the model can be made to say or request anything an attacker wants — the architecture's job is to make that harmless, not to make it impossible for the model to be fooled.

For the actual attack techniques these controls defend against, and how to red-team them, go to Module 13 — AI Security [10-ai-architecture/13-ai-security]. This section only answers "where," on purpose.

4. Deployment patterns

4.1 A prompt change is a deploy

Treat every prompt edit like a code change, because it has the same blast radius: it can silently change behavior for 100% of traffic the moment it ships. That means: version control, review, a rollout strategy, and a rollback path — not a hotfix typed into a config UI at 5 p.m.

4.2 Online vs. batch/async inference

	Online (real-time API)	Batch / async
Latency requirement	seconds, user is waiting	minutes to hours acceptable
Cost	full price, no batching discount	many providers discount batch APIs 50%+
Use for	chat, live agent turns, anything user-facing synchronously	bulk classification, nightly re-scoring, embedding backfills, report generation
Architecture fit	model gateway, streaming (§1.5)	the queue/worker pattern from §1.1, no SSE needed

Route work to batch whenever the user isn't watching a clock. It's the least glamorous cost optimization in this module and one of the largest — a nightly re-embedding job or a bulk-classification pass has no business going through the same low-latency, full-price path as a live chat turn.

4.3 Blue-green and canary deploys — for prompts and models, not just code

Strategy	Mechanism	Best for
Blue-green	two full environments; flip traffic at a router/load-balancer, instant rollback	a model or infra version bump where you want an instant, total rollback
Canary	route a small % of traffic (by request, or by tenant) to the new prompt/model version; watch metrics; ramp	a prompt change, where you want to catch a regression on 1% of traffic, not 100%
Shadow deployment	send a copy of production traffic to the new version, discard its output, compare against production's actual response	testing a new model/prompt against real traffic with zero user-facing risk — no output is ever served

Shadow deployment deserves emphasis because it's underused: before a new model version or a significant prompt rewrite goes anywhere near a canary, mirror real production requests to it, log both outputs side by side (or run an LLM-as-judge diff, Module 15 [10-ai-architecture/15-ai-evals]), and quantify the delta on real traffic distribution — not your eval set, which is smaller and staler than production traffic by definition. Only promote to canary once shadow results are acceptable.

4.4 The model/prompt registry pattern

A registry is the thing that makes "which prompt/model produced this response, and can I roll back to the one that worked yesterday" answerable in seconds instead of a git-archaeology exercise:

- Every prompt template and its associated model/parameters gets a version identifier (a commit hash is sufficient; a bare mutable name like "production" is not — see the caution in Module 16 / langsmith [10-ai-architecture/16-ai-tech-stack/tracks/langsmith/notes/02-evaluation-and-operations.md] about pinning a commit or tag rather than a floating pointer).
- Every logged request records the exact version used (this is the observability layer, §2, feeding the registry's audit trail).
- Rollback is "point the router at the previous version," not "redploy."

Why auditability requires this, not just convenience: if a regulator, an internal auditor, or a customer asks "what exact system produced this decision on this date," the registry plus the trace store (§2) is the only honest answer. Module 12 — Governance [10-ai-architecture/12-ai-governance] treats this pairing as a hard requirement for any system making decisions that affect people, not a nice-to-have.

5. Cost architecture — from cost model to actual decisions

Module 04 [10-ai-architecture/04-llm/notes/02-inference-and-adaptation.md] gives you the cost *model* (tokens × price, caching saves X%, agents multiply cost by turn count). This section is where that model becomes architecture decisions: three specific components, placed in three specific layers.

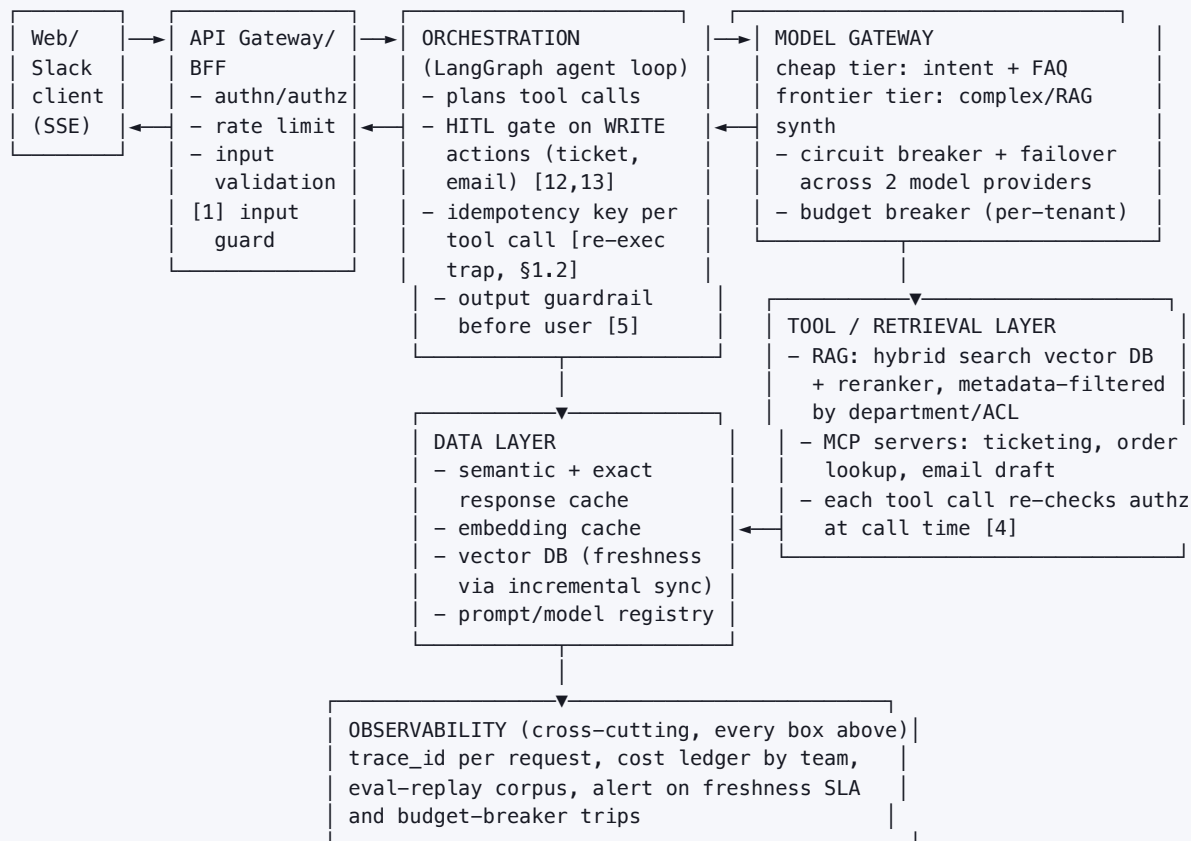
Worked example — target: a support-assist feature at \$0.02 median cost per resolved conversation, 1M conversations/month, hard ceiling \$30k/month.

Decision	Where it lives	Why
Cost-tiered router: 85% of turns (simple FAQ-shaped) → small/cheap model; 15% (complex, multi-turn) → frontier model	Model layer (gateway)	Cuts blended cost ~5-8x vs. always-frontier, per the routing math in Module 04 [10-ai-architecture/04-llm/notes/02-inference-and-adaptation.md]
Exact-match + semantic cache in front of the top ~200 recurring FAQ intents	Data layer, called from orchestration before any model call	These intents are high-volume and low-stakes enough to accept the semantic-cache trade-off from notes/01 §4.1 [10-ai-architecture/notes/01-reference-architecture.md]
Prompt template built stable-prefix-first (system + tool schema + FAQ few-shot, then retrieved context, then user turn)	Orchestration layer's prompt builder	Makes provider-side prefix caching actually apply — free money if you order it right, wasted if you don't
Hard per-tenant and global budget breaker, checked before the call, not after	Model layer (gateway), fed by the cost ledger (§2)	Turns "we found out at month-end" into "the 1,000,001st cheap-tier request got throttled at \$29,800"
Escalation path when budget breaker trips: degrade to cheapest tier only, don't fail the request	Model layer, per the graceful-degradation table in §1.3	Protects the budget without taking the product down

The pattern that generalizes: **every line item in a cost model from Module 04 corresponds to exactly one architectural decision — a router, a cache, a prompt structure, or a breaker — and that decision has exactly one correct layer to live in.** If you can't point at the layer, the cost control doesn't actually exist yet; it's still a spreadsheet.

6. Capstone reference architecture

System: an internal enterprise assistant — employees ask questions against company docs (RAG) and can trigger real actions (file a ticket, look up an order, draft an email) via tools (MCP servers).



Component-by-component rationale

- **Client (SSE)** — chosen over WebSocket per §1.5: employees read streamed answers, don't need mid-stream interrupts. Holds no secrets; auth token only.
- **API gateway/BFF** — the only layer employees' browsers/Slack talk to; owns per-user rate limits (an internal tool still needs them — one runaway script can drown the model budget) and is control point [1] in the security request-flow (§3).
- **Orchestration (LangGraph agent loop)** — owns planning and, critically, the **human-in-the-loop gate before any write action** (filing a ticket, sending an email) — read actions (RAG lookups) don't need approval, write actions do, because a wrong read is an annoyance and a wrong write is an incident. Every tool call carries an idempotency key so a resumed/retried step doesn't re-file the ticket — this is the re-execution trap from §1.2, and it is *exactly* the failure mode this system would hit without the key, since HITL approval means every write action, by construction, involves a pause-and-resume.
- **Model gateway** — two providers behind one interface, with the cost-tiered split from §5's worked example (cheap tier handles intent classification and simple FAQ synthesis; frontier tier handles multi-document RAG synthesis and anything the cheap tier's confidence score flags). Circuit breaker and per-tenant budget breaker both live here, per notes/01 §3 [10-ai-architecture/notes/01-reference-architecture.md].
- **Tool/retrieval layer** — RAG uses hybrid search (employees search for exact ticket numbers and product SKUs as often as natural-language questions) with metadata filtering by department ACL, so retrieval itself enforces "you can only retrieve documents you're allowed to see" — this is a security control (§3, control

4) implemented as a data-layer feature, not a prompt instruction. MCP servers expose the ticketing/order/email tools with their own argument-schema validation independent of what the model claims it's calling them with.

- **Data layer** — the semantic cache sits only in front of the FAQ intents that were shadow-tested and shown safe under §4.1's threshold trade-off; the RAG index uses incremental content-hash sync (§5.1 of notes/01) with a freshness SLA per document source, alerted on by observability.
- **Observability** — a single `trace_id` threads through every box above, feeding the cost ledger (by team, for chargeback), the eval-replay corpus (production traces become tomorrow's regression tests, Module 15 [10-ai-architecture/15-ai-evals]), and the freshness/budget alerting that makes the graceful-degradation and budget-breaker paths in §1.3 and §5 actually observable when they fire.

Every box in this diagram is justified by a specific pattern earlier in this module or the previous note — that is the test for whether your own architecture diagram is done: can you point at a reason for every box, or is a box there because a tutorial had one?

7. Forward/backward links

Idea here	Where it returns
Circuit breaker / failover mechanics	notes/01-reference-architecture.md §3 [10-ai-architecture/notes/01-reference-architecture.md], code/model_gateway_simulator.py [10-ai-architecture/code/model_gateway_simulator.py]
Re-execution trap, idempotency	Module 16 / langgraph [10-ai-architecture/16-ai-tech-stack/tracks/langgraph/notes/02-production-craft.md]
Tracing implementation detail	Module 16 / langsmith [10-ai-architecture/16-ai-tech-stack/tracks/langsmith]
Prompt-injection mechanics and red-teaming	Module 13 — AI Security [10-ai-architecture/13-ai-security]
Prompt registry, versioning, rollback in practice	Module 16 / langsmith §7 [10-ai-architecture/16-ai-tech-stack/tracks/langsmith/notes/02-evaluation-and-operations.md]
Auditability, human-oversight obligations	Module 12 — AI Governance [10-ai-architecture/12-ai-governance]
Regulatory evidence requirements on top of auditability	Module 14 — AI Compliance [10-ai-architecture/14-ai-compliance]
Eval-replay corpus, LLM-as-judge, regression suites	Module 15 — AI Evaluation [10-ai-architecture/15-ai-evals]
Cost model this section turns into decisions	Module 04 §6 [10-ai-architecture/04-llm/notes/02-inference-and-adaptation.md]